Project 3: Make a Memory Game with React

NOTE: *The assignments in this course use HTML5 and CSS3.*

Are you ready to have a little fun?!

I thought we would wrap up this course by creating a **Memory Game** app *(by Net Ninja)* using React. You will upload your completed game to a **GitHub repository** within your GitHub account as well as upload the contents of your **production build** to the Liquid Web server.

This wonderful tutorial series contains 11 short videos, code walkthroughs which you will follow along with, and explanations. At the end of the video tutorial series, you should have a functioning memory match game. Please allow yourself a minimum of 2-3 hours to complete the tutorial. Additional time may be required for testing, debugging, and tweaking.

Section 1.0 – Objectives & Requirements

The objective of the **Memory Game** app that you are building is to randomly display 12 cards on screen containing 6 different fruit images. The player clicks on a square to reveal an image and tries to locate the matching image. If there is a match, the cards remain visible on screen. If there is not a match, the cards flip back over. The number of turns to complete the game and match all the images, is recorded. A New Game button is present that restarts the game.

You will work with the following while building the app:

- Functions
- Components
- Arrays
- Various methods
- CSS
- Images
- The useEffect hook
- The package.json file
- And more…

You will use Node.js, Visual Studio Code (VS Code), the Terminal, and a web browser to complete the assignment. You will also be placing your project under Source Control and uploading it to your **GitHub** account as well as uploading a **build folder** to your account fold on the Liquid Web server. Please use the instructions in Module 10 regarding GitHub, to assist you with these requirements.

Happy Coding!

Section 1.0 – Downloading the Starter Files & Basic Setup

A **Project 3 Discussion Forum** is set up within the Module 11 folder specifically for you to ask questions of each other, collaborate, get feedback, offer advice and/or solutions. This discussion forum should be busy and hopping with ideas and advice from each other, it's your forum to use. This is such as valuable resource; I can't stress that enough.

Open up your favorite web browser, head on over to https://tinyurl.com/2exn5t4y, and let's have some fun!

NOTE: Visual Studio Code (VS Code) is the recommended IDE for this course.

1. Begin by jumping ahead to around the **4:51** mark of the **Build a Memory Game with React #1 – Intro & Setup** video in the playlist. You will be walked through how to download the starter files from GitHub.
2. The starter files for this project can be found on the **Net Ninja** GitHub page at https://github.com/iamshaunjp/React-Firebase/.
3. Download the starter zip file from GitHub to your personal computer and extract the contents of the folder.

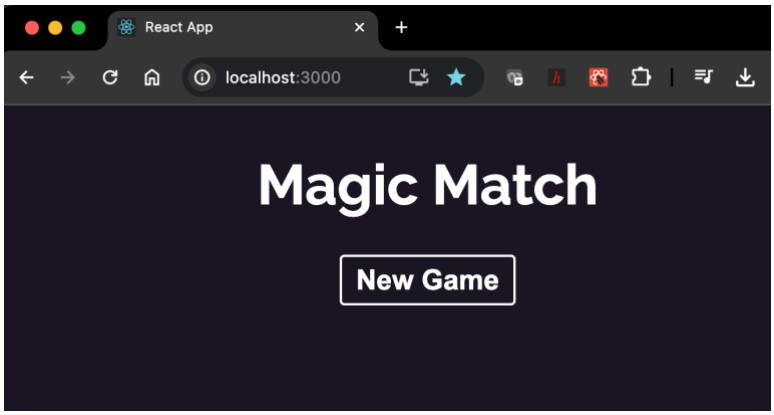
We will be using different images for this project that are not copyrighted images and are free to use from Pixabay.

4. Download the **matchinggameimages.zip** file and extract its contents.
5. Locate the magic-memory folder from the steps above and navigate to the img folder.
6. Replace the **matchinggameimages** with the ones provided. You don't want any copyrighted images within your img folder.
7. Open the magic-memory folder in VS Code and proceed to through the rest of the setup video.
8. Once all dependencies finish installing, locate the **package.json** file and add the following:
    a. Locate the **version** field ("version": "0.1.0") near the top of file and directly after the comma, hit the enter/return key to move to the next line and type the following (don't forget the comma):

    "homepage": ".",

    b. Save the package.json file.

9. After everything is completely installed, go to the Terminal in VS Code and type **npm run start** to compile your project. Once it compiles, it should open up in your local browser window and look like the screenshot below at this point.



Section 1.1 – Working through the Playlist

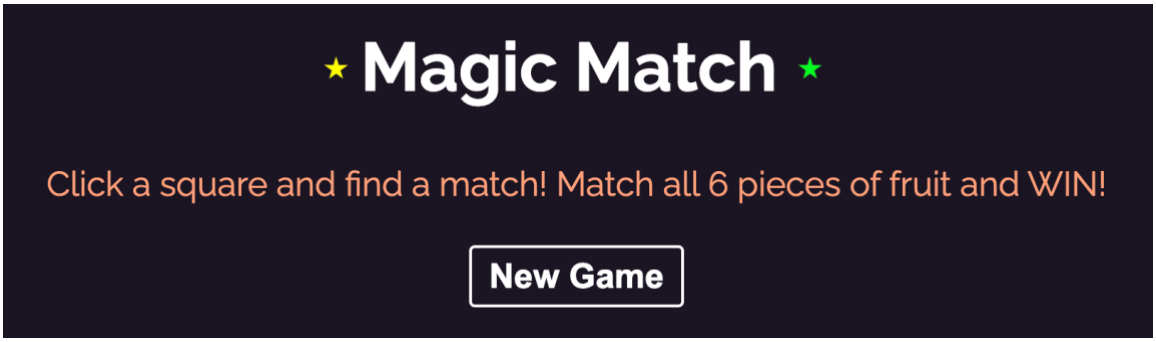
Be sure that you have the new images that were supplied to you for this project. You will need to make the necessary image name changes to reflect the new images used for this project that I have supplied to you.

As you work your way through the video playlist, please keep the following in mind:

- Watch your syntax.
- Test as you go along.
- The **Developer Tools** in your web browser are extremely useful!
- Don't rush through the video and code, take your time to fully understand what the code is doing.
- Be cognizant of what file you are adding your code to.
- Add comments! This is very useful if you need to go back and debug your code.

***PLEASE READ*:** After working through the project, creating a production build, and uploading the build to the web server, the images displayed as missing when viewed within a web browser. It looks fine locally, but when you type in the URL to your build on the server, they are missing. The solution lies in the file path. Removing the initial / for each image in App.js and the image in SingleCard.js, corrects the issue.

This is not in the video, but you will include a line or two of basic instructions. This should be added to your App.js file and styled using CSS. As an example, this is what I did:

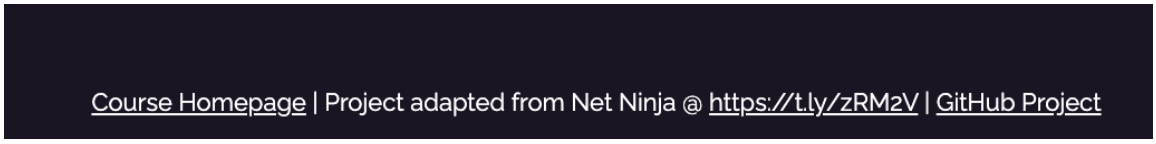


I also added the **small stars** on each side of the heading (not in the video) using CSS and the character entity code for a star to display them (it's not an image). Feel free to embellish the design if you wish, but keep in mind that it has to function properly and should maintain readability.

The Footer Stuff

You will **NOT** be including HTML or CSS validation links in your Project 3 app footer.

This is not in the video but you will include a footer section in your app that contains the following as shown below: a link to your **Course Homepage**, the text indicating where the **project was adapted from along with the YouTube URL**, and a link to your **GitHub Project**. Beside yourself, I should be the only one that should be a collaborator on your project, so I should be able to click the link and access your work. I styled my example using CSS with a font size of 10px. I also made the text and links all white. Remember, readability on a dark background.



Section 1.2 – Creating a Production Build

When you have fully tested your project and everything is in working order, run a **production build** and create a **build** folder (Page 161).

1. Within your terminal window, type the following command (Page 161):

```
npm run build
```

Next, you will be uploading the **contents** of the build folder to your account folder on the web server.

Please remember that each time you modify code and save it, you should rerun the production build otherwise you will not have the latest and greatest code to upload.

Section 1.3 – Uploading to the Liquid Web Server

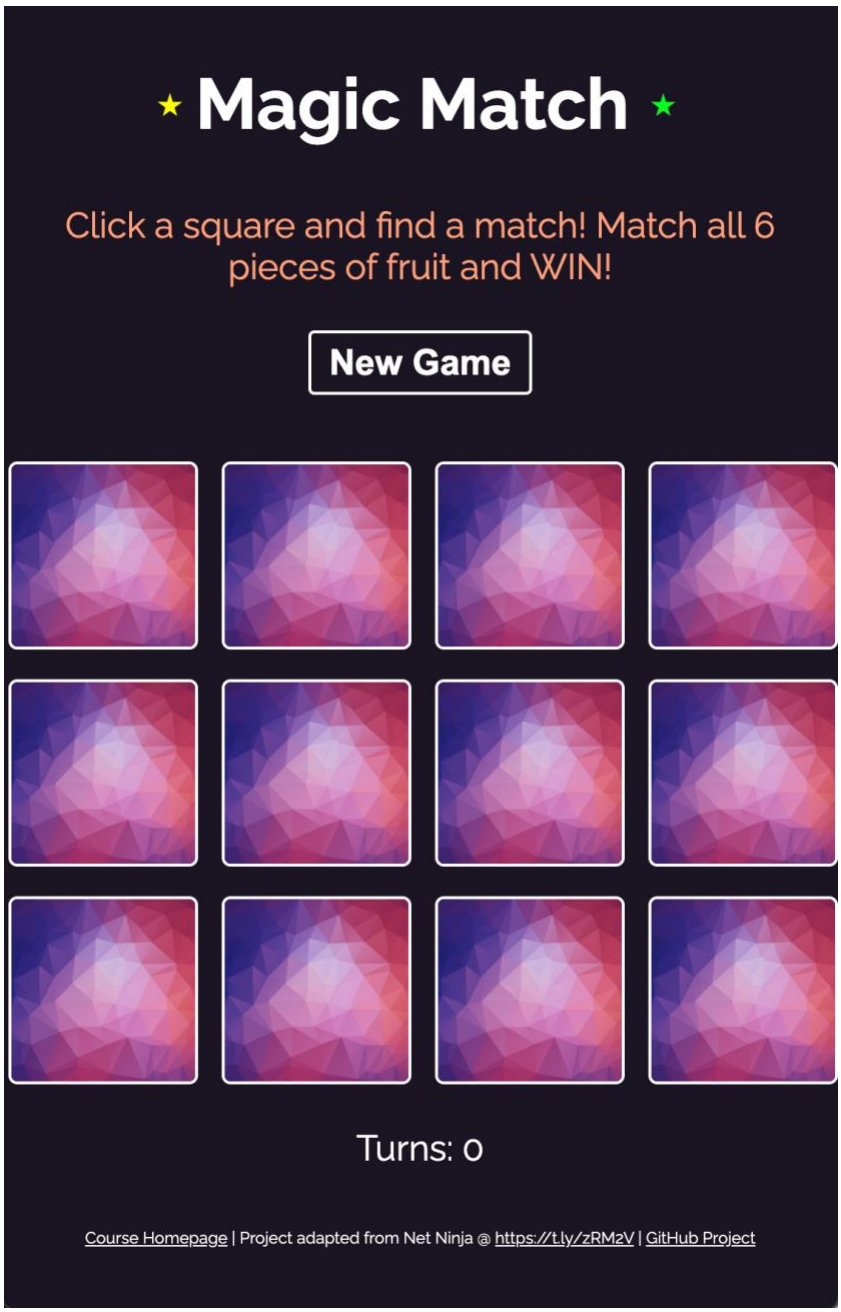
Since a fully completed project folder that includes the node_modules folder, is quite large in file size, we are going to only upload the build folder which is essentially compressed and minified files.

1. Navigate to the project3 folder in your local environment and locate the build folder.
2. Launch FileZilla, connect to your account folder on the web server, navigate to the itwp1150 folder, create a project3 folder (or whatever you want to name it, but **do not** include spaces in the folder name), and upload the **contents** of the build folder into the homework folder you created.

3. Once the contents of the build folder have been uploaded, test your web page by typing in the URL to the index.html file within the homework folder you created. For example, if I created a folder titled **project3** for this assignment, my testing/linking URL would be:

   http://jwanner.macombserver.net/itwp1150/project3/index.html

The React project should now display within your browser window. This is a screenshot of my completed game as displayed on the web server:



Section 1.4 – Uploading to GitHub

Similar to uploading your Homework 8 assignment to GitHub, you will also be uploading your completed Project 3 to your GitHub account **and adding me as a collaborator**. The same instructions provided within your Homework 8 assignment are repeated below for your convenience.

1. Please watch the video I put together that walks you through how to accomplish this at https://macomb.techsmithrelay.com/GfB4.
2. Please walkthrough the **Setting up your GitHub Repository & Uploading to GitHub Tutorial** within this module's folder for instructions on how to create your repository in GitHub and upload your Project 3 assignment files to your GitHub account.
3. Update your course homepage to link to your Project 3 GitHub repository within your GitHub account and the index.html file within the build folder for Project 3 on the web server.

NOTE: Your course homepage should have two hyperlinks for this assignment. One that links to your Project 3 GitHub repository and one that links to Project 3 on the Liquid Web server. Be sure to add me as a collaborator for Project 3.

Section 1.5 - Submitting your Work

Please note that you always need to notify me that your work is ready to be graded through the appropriate assignment area in Canvas each week. Missing notifications will not receive credit.

1. Log in to the Canvas classroom.
    a. Click on the **Home** tab in the side navigation bar.
    b. Click on **Module 12** and locate the **Module 12 – Project 3** link.
    c. In the comments area let me know you are finished and your assignment is ready for grading, "my work is ready for grading." will do just fine.
    d. **Supply me with your course homepage URL.**

    e.g., http://verlanderj456@macombserver.net/itwp1150/home.htm

    e. Please note that you will not be attaching your completed assignment file(s) to the assignment area but rather uploading it to the web server where I will access it. It is not necessary to attach any files. **The file attachment option has been disabled since you will be uploading your work to the student web server.**
    f. Once you have submitted notification and your work has been graded, you cannot resubmit for a better grade, the original grade remains as is.

Please be sure to test the links to your work. Test access to your assignment from your course homepage to be sure everything is correctly hyperlinked and in working order. If I cannot access your work, I cannot grade it. The result will be a zero for the assignment.

Please review the Weekly Schedule for other required assignment(s) due this week.

Grades will be assigned within the limits of the grading policy. The use of correct syntax within coding assignments is implied. Points listed within the Value column of the rubric below are the maximum number of points you can receive if your work is 100% correct. Incorrect work may receive zero through the maximum point value range listed within the Value column. ***Late work is not accepted. The appropriate assignment area(s) will be closed at the time of assignment's due date. Email attachments of any assigned submitted work will not be accepted.***

Please review the course Late Policy within Syllabus (First Day Handout).

Project 3 Grading Rubric

Grading Criterion

Criterion	Value
Created and updated HTML index build file containing the basic structural tags including the meta tag specifying the UTF-8 character set and an appropriate title within the title tag. The English language attribute for the web page must be present within the HTML tag along with the following links: Course Homepage, GitHub Project, and the Project mention with the linked YouTube video.	10
Created external CSS documents containing the style rules for formatting the components and overall look of the app.	10
Created JavaScript files that contain the required and necessary imports, export component, components, state information, and houses the programming logic for the app.	15
Created components folder is present that contains the SingleCard js and CSS files.	5
Fruit images and card cover image are randomly generated, present, and displaying correctly within the app.	10
Instructions are present for playing the app.	5
The number of turns functions and displays correctly throughout the app as it's being played.	10
The New Game button functions and displays correctly within the app. When clicked, a new game begins.	10
The completed React application must function, display correctly, and include all of the tutorial features, animations, included instructions when viewed within a web browser based on the project requirements.	30

A production build was completed and the build folder contents are present within a designated folder on the web server. A link to the index.html file within your build folder must be present within your course homepage.	25
Your completed assignment resides within a GitHub repository in your GitHub account and is accessible. A link to your GitHub repository must be present within your course homepage and the footer of your app.	25
TOTAL POSSIBLE POINTS:	**155**